Teaching an AI to smell: how it works and how to build it
Hari (@harixganesh)

Summary

In 2023, researchers at Google Research and the Monell Chemical Senses Center published a model in Science that predicts what a molecule smells like from its structure, as words (fruity, mint, smoky, fishy).

Your nose has about 400 types of smell receptors, and each molecule switches on a different mix that your brain reads as vanilla or rotten eggs. The model skips the nose and goes straight from structure to words. A free community copy, **OpenPOM**, lets you train one yourself, and a cheap gas sensor can learn a few real smells.

How it works

1. Turn the molecule into dots and lines. Molecules are written as short strings called **SMILES**. Ethanol (drinking alcohol) is CCO: carbon, carbon, oxygen (hydrogens left out). As a **graph**, atoms are the dots (**nodes**) and bonds are the lines (**edges**), so ethanol is 3 nodes and 2 edges. Each atom gets a few numbers, its **features**. That's all the computer is given:

Atom	Element	Bonded to	Charge	In a ring
1	carbon	1 atom	0	no
2	carbon	2 atoms	0	no
3	oxygen	1 atom	0	no

2. Give it the answers. Each molecule comes with its **labels**: a 1 or a 0 for each of 138 smell words. A molecule can have several at once (**multi-label**). Two real rows:

Molecule (SMILES)	fishy	rose	sweet	cinnamon	...138 words
CC(O)CN	1	0	0	0	...
O=C(O)CCc 1ccccc1	0	1	1	1	...

The labels come from fragrance and flavour catalogues, so they're a bit noisy.

3. Let the atoms talk. The model is a **message passing neural network (MPNN)**, a type of **graph neural network**. A **neural network** is a function with lots of adjustable numbers inside (**weights**). Each round, every atom mixes in its bonded neighbours' numbers. On ethanol, after round 2 the first carbon has heard about the oxygen, two bonds away. OpenPOM runs 5 rounds, then combines the atoms into one list of numbers.

4. The odor map and the scores. A few more layers follow. The second-to-last, 256 numbers, is the **principal odor map (POM)**: molecules that smell alike get similar numbers. The last layer gives 138 scores, one yes-or-no per word.

5. Train it, then test it. **Training** means nudging the weights whenever its scores miss the labels. Then you score it on a **test set** of molecules it never saw, with **ROC-AUC**: in every pair of one fruity and one non-fruity molecule, how often does the fruity one score higher? 4 fruity and 6 not is 24 pairs; 21 right is 21 / 24 = 0.875. 0.5 is a coin flip, 1.0 is perfect.

On about 400 new molecules rated by a panel of at least 15 trained smellers, the model was closer to the panel's average than a typical single panelist on 53% of them. One real limit: it predicts what a smell is like, not how strong, and only for single molecules, not mixtures.

The sensor is a separate model: a Bosch **BME688** chip reads its electrical **resistance** (how hard current gets through) at 10 heater temperatures, so each smell gives a pattern of 10 numbers. You record a few smells you can repeat (coffee, plain air) and train a **classifier**, a model that picks one of your labels, without OpenPOM.

How you would make one

- Open a free Google Colab notebook and install OpenPOM.
- Load the 4,983 labelled molecules and split them into training, validation and test sets.
- Train the graph network and score it on the test molecules.
- Type in a molecule and see which smell words it predicts.
- Optional: get a BME688 board, record a few smells and train it to tell them apart.

The model half is free in Colab, Google's online notebook with a free GPU (not guaranteed), and you need basic Python. The sensor half needs a $20 board, a Raspberry Pi and a Windows machine.

That's the gist. If you actually want to build it, keep going.

If you actually want to build it

1. Set up in Colab. Open a new notebook, switch the runtime to a GPU, and run the install lines from OpenPOM's README in order: a pre-release DeepChem (the chemistry library OpenPOM plugs into), PyTorch 2.4.0, DGL (the graph library) from DGL's own download page, then openpom. It worked when importing openpom gives no error. (Not your laptop: those versions only exist for Linux on Intel or AMD chips, which is what Colab is.)

2. Load and split the data. The molecules are already in the repo (openpom/data/curated_datasets, the file ending in 4983.csv). The training notebook, examples/mpnn_pom_train.ipynb, loads them with a **featurizer**, which turns each SMILES into the atom graph. Then it splits them 80/10/10 into training, **validation** (molecules you check while tuning) and test. Its iterative stratified splitter is built for multi-label data, so rare words land

in every split, which matters because fruity has 1,902 molecules and smoky 72. That gap is **class imbalance**, and the notebook weights each word so the rare ones count.

3. Train it. The notebook builds MPNNPOMModel with the paper's design: 5 rounds of message passing, the 256-number map, **dropout** of 0.12 (switching off a random 12% of numbers in training so it can't memorise) and a **learning rate** (how big each nudge is) that shrinks. These are **hyperparameters**: you choose them, it doesn't learn them. It trains only 5 **epochs** (passes through the data); the paper used 150. (Run the 5 first, then raise it.)

4. Make it predict. The demo script, predict_odors.py, takes a SMILES like CCO and charts its top smell words. It needs an **ensemble** of 10 trained models (trained separately, scores averaged, steadier than one), so train them first with the ensemble notebooks in examples.

5. Optional: give it a nose. Buy a BME688 breakout board (Adafruit's is $19.95). Run a new board for a day or two first, and warm it up about 30 minutes each session. Record each smell several times in a sealed container on a Raspberry Pi (PiCockpit tutorial below). Import the recordings into Bosch's BME AI-Studio, make one class per smell, train, and export the result to the board. (AI-Studio is Windows only.)

How you know it works

The model: ROC-AUC on the test set, averaged across the words. 0.5 is guessing. Full training with an ensemble gets about 0.89, so don't expect that from the demo.

The sensor: accuracy on recordings it never trained on, and the **confusion matrix** (what it guessed against what it really was). Above 80% is worth putting on the board, above 90% is good, and a perfect score usually means too little data.

Where people mess up

- Looking for a trained model to download. The repo has none. You train it.
- Expecting the sensor to name a new smell. It calls anything new the closest smell it knows.
- Testing the sensor on recordings from the same sitting it trained on. Humidity and the sensor drift, so record on different days.

Start here

In this order:

- Google Research: Learning to Smell. The idea in pictures.
- DeepChem: The Basic Tools of the Deep Life Sciences. Your first DeepChem model.
- OpenPOM on GitHub. The README, the data and the training notebook.
- A principal odor map unifies diverse tasks in olfactory perception. The paper, free.
- PiCockpit: Teach your BME688 how to smell. The sensor half, with Bosch's docs.

If you build it, send me what you get!